

Programming Problem Description

-----Task description-----

```
def numDistinct(self, s: str, t: str) -> int:
```

""""Given a string S and a string T, count the number of distinct subsequences of S which equals T. A subsequence of a string is a new string which is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (ie, "ACE" is a subsequence of "ABCDE" while "AEC" is not). """"

-----Example1-----

Input: S = "rabbbit", T = "rabbit"

Output: 3

Explanation:

As shown below, there are 3 ways you can generate "rabbit" from S.

-----Example2-----

Input: S = "babgbag", T = "bag"

Output: 5

Canonical Solution

```
def numDistinct(self, s, t):
    setOfT=set(t)
    news=""
    for ch in s:
        if ch in setOfT:
            news+=ch
    dp=[[1 for i in range(len(news)+1)] for j in range(len(t)+1)]
    for j in range(1,len(t)+1):
        dp[j][0]=0

    for i in range(len(t)):
        for j in range(len(news)):
            if t[i]==news[j]:
                dp[i+1][j+1]=dp[i][j]+dp[i+1][j]
            else:
                dp[i+1][j+1]=dp[i+1][j]
    return dp[len(t)][len(news)]
```

Unit Test & Meta Data

```
"inputs": [
  [
    "\"rabbbit\"",
    "\"rabbit\""
  ]
],
"outputs": [
  3
],
"fn_name": "numDistinct",
"starter_code": "\nclass Solution:\n    def numDistinct(self, s: str,\nt: str) -> int:\n"
```